

Programação Orientada a Objetos



Tipo Abstrato de Dados

Tipo Abstrato de Dados

Há dois recursos que ajudam a lidar com a complexidade inerente ao desenvolvimento de códigos, facilitando sua alteração e manutenção:

- **Decomposição:** quebrar um problema em partes menores (classes/módulos)
- **Abstração:** expor só o essencial e suprimir detalhes

Decomposição e abstração criam uma estrutura clara e reduzem complexidade.

Tipo Abstrato de Dados

Tipo abstrato de dados (TAD): especificação lógica de um conjunto de valores (objetos) e das operações válidas sobre eles, incluindo o comportamento dessas operações (o que fazem, pré/pós-condições e efeitos), sem definir como são implementadas.

Tipo Abstrato de Dados

- **Contrato/Interface**: específica o que cada operação faz (efeitos, entradas/saídas)
- **Implementação:** define como fazer (estruturas de dados, algoritmos)
- **Objetivo**: permitir mudar a implementação sem quebrar o código de quem usa o TAD



Definindo POO

Definindo POO

- Paradigma que organiza o software em objetos que combinam estado (dados) e comportamento (métodos).
- Quatros pilares da POO e por que usar:
  - **Abstração**: focar no essencial, ocultando detalhes irrelevantes.
  - **Encapsulamento:** proteger o estado com uma interface estável (ex.: @property).
  - **Herança:** reutilizar/completar comportamento de uma classe base.
  - **Polimorfismo:** a mesma interface com múltiplas implementações.

Definindo POO

- **Encapsulamento:** esconde detalhes internos e expõe uma API clara.
  - **Application Programming Interface (API):** conjunto de regras e pontos de acesso (funções, métodos, URLs, mensagens) que permite que um software converse com outro sem expor os detalhes internos.
  - Pense nela como a interface pública de uma biblioteca/classe/serviço.

Definindo POO

- Conceitos importantes:
  - **Classe:** molde/“tipo” que descreve quais atributos e métodos os objetos terão.
  - **Objeto (instância)**: realização concreta de uma classe.
  - **Atributos:** dados do objeto; onde **métodos** são funções que operam nesses dados.
  - **Mensagens:** objetos interagem chamando métodos uns dos outros (acoplamento controlado).



Classes, Objetos e Atributos

Classes, Objetos e Atributos

- **Classe**: “molde” (tipo) que define dados + operações

```
2   class Ponto(object):
3
4       def __init__(self,x,y):
5           self._x = x
6           self._y = y
7
8       def distancia(self,outro):
9           deltaX=(self._x-outro._x)
10          deltaY=(self._y-outro._y)
11          return (deltaX**2+deltaY**2)**0.5
12      def __str__(self):
13          return '('+str(self._x)+','+str(self._y)+')
```

Classes, Objetos e Atributos

- **Objeto/Instância**: ocorrência concreta da classe

```
2   class Ponto(object):
3
4       def __init__(self,x,y):
5           self._x = x
6           self._y = y
7
8       def distancia(self,outro):
9           deltaX=(self._x-outro._x)
10          deltaY=(self._y-outro._y)
11          return (deltaX**2+deltaY**2)**0.5
12      def __str__(self):
13          return '('+str(self._x)+','+str(self._y)+')
```

```
p=Ponto(3,4)
print(p)
q=Ponto(4,8)
print(p,q)
print(p.distancia(q))
print(q.distancia(p))
```

```
(3,4)
(3,4) (4,8)
4.123105625617661
4.123105625617661
```

Classes, Objetos e Atributos

- **Atributos:**
  - **Atributos de classe:** compartilhados por todas as instâncias
  - **Atributos de instância:** pertencem a cada objeto

```python
class Pessoa:
    # atributo de classe (o mesmo para TODAS as instâncias)
    especie = "Homo sapiens"

    def __init__(self, nome, idade):
        # atributos de instância (cada objeto tem os seus)
        self.nome = nome
        self.idade = idade

a = Pessoa("Ana", 30)
b = Pessoa("Bruno", 25)

print(a.especie, b.especie)     # Homo sapiens Homo sapiens
print(a.nome, b.nome)           # Ana Bruno (próprios de ca
```

```
Homo sapiens Homo sapiens
Ana Bruno
```

Classes, Objetos e Atributos

```python
class Contador:
    criados = 0  # de classe

    def __init__(self):
        Contador.criados += 1   # atualiza o compartilhado
        self.id = Contador.criados  # de instância

c1 = Contador()
c2 = Contador()
c3 = Contador()

print(Contador.criados)  # 3 (valor único para a classe)
print(c1.id, c2.id, c3.id)  # 1 2 3 (cada instância tem seu id)
```

```
3
1 2 3
```

Classes, Objetos e Atributos

- Objetos são instâncias de uma classe.

  p=Ponto(3,4) ⇒ criou-se o objeto p como uma instância da classe Ponto.
  print(p.x,p.y) ⇒ imprime 3 e 4, valores dos atributos x e y do objeto p, **se x e y não estão definidos como protegido (self._x e self_y)**
  q=Ponto(4,8) ⇒ outro objeto da classe Ponto.
  d= p.distancia(q) ⇒ calcula distância entre p e q

Classes, Objetos e Atributos

- Para acessar diretamente um dado de um objeto:

  <objeto>.<atributo> ⇒ p.x ou p.y

  Isso não será possível se a classe definir como protegidos self._x e self_y

- Para acessar um método de um objeto:

  <objeto>.<método> ⇒ p.distancia(q)

Classes, Objetos e Atributos

- Nós já estávamos manipulando objetos em Python:
  - L=[10,20,30] é um objeto que instancia um tipo lista.
  - L.append(40) acessa append() que é um método de lista.
  - ‘João’ é uma instância do tipo string.
  - 2.5 é uma instância do tipo float.

Classes, Objetos e Atributos

- Utilizando o paradigma da POO podemos
  - Criar objetos de um novo tipo definindo novas classes.
  - Manipular tais objetos
  - Descartar tais objetos.

Classes, Objetos e Atributos

- Classes podem ser vistas como abstrações para atributos de dados
- As especificações das operações executadas em um classe estabelecem uma interface entre um tipo abstrato de dados e o resto do programa.
- A interface estabelece o comportamento das operações, ou seja, o que elas devem fazer….mas não como elas fazem..
- As classes facilitam a reutilização de código.

Classes, Objetos e Atributos

- Criando uma classe: **class** <nome da classe>**(**<classe pai>**) :**
- **class** Ponto(**object**):
  - Ponto herda todos os atributos da classe **object** definida em Python.
  - Ponto é uma subclasse de **object**
  - **object** é uma superclasse da classe Ponto.

Classes, Objetos e Atributos

- Os atributos de uma classe são representados pelos:
  - dados : x e y na classe Ponto
  - Métodos

```
def distancia(self,outro):
    deltaX=(self._x-outro._x)
    deltaY=(self._y-outro._y)
    return (deltaX**2+deltaY**2)**0.5
```

Classes, Objetos e Atributos

- Um objeto é instanciado através de uma chamada ao método **__init__**.
- O método **__init__** inicia alguns dados do objeto.

```
def __init__(self,x,y):
    self._x = x
    self._y = y
```

Classes, Objetos e Atributos

- self: referência explícita à instância nos métodos
- Convenção: usar self como primeiro parâmetro de métodos

```
def __init__(self,x,y):
    self._x = x
    self._y = y
```

```
def distancia(self,outro):
    deltaX=(self._x-outro._x)
    deltaY=(self._y-outro._y)
    return (deltaX**2+deltaY**2)**0.5
```

```
def __str__(self):
    return '('+str(self._x)+','+str(self._y)+')'
```

Classes, Objetos e Atributos

```
def __init__(self,x,y):
    self._x = x
    self._y = y
```

- **self** se refere ao próprio objeto instanciado.
- p=Ponto(3,4)
  - Ponto(3,4) chama **_init_**(**self**,3,4)
  - **self** não precisa entrar como argumento em Ponto(3,4).

Classes, Objetos e Atributos

```
def distancia(self,outro):
    deltaX=(self._x-outro._x)
    deltaY=(self._y-outro._y)
    return (deltaX**2+deltaY**2)**0.5
```

- **self** se refere ao próprio objeto instanciado.
- outro é o argumento relativo ao outro objeto instanciado
- d=p.distancia(q)
    - **self** se refere ao objeto daquele método, ou seja, p.

Classes, Objetos e Atributos

Implementar vs utilizar classes

Implementar

- Estabelecer **o que é a classe**.
  - Há atributos de dados que caracterizam a classe.
- Estabelecer **o que a classe faz**.
  - Há métodos que caracterizam as funcionalidades da classe

Utilizar

- Criar instâncias de um objeto.
  - Define-se um novo tipo de objeto a ser manipulado.
- Manipular tais objetos
  - Executar operações com tais objetos.

Classes, Objetos e Atributos

Implementar vs utilizar classes

- A classe contém a definição de dados e métodos comuns a todas as instâncias da classe.
- O nome da classe define um novo tipo

```
class Ponto(object)
```

- `self é um parâmetro para acessar atributos relativos a uma instância da classe (objeto).`

```
self.x=x
```

Classes, Objetos e Atributos

Implementar vs utilizar classes

- A instância de uma classe é um objeto específico

```
p = Ponto(3,4)
```

- Duas instâncias de uma mesma classe podem apresentar valores diferentes

```
p = Ponto(3,4) q = Ponto (2,1)
```

- A estrutura de duas instâncias de uma mesma classe é a mesma.

Classes, Objetos e Atributos

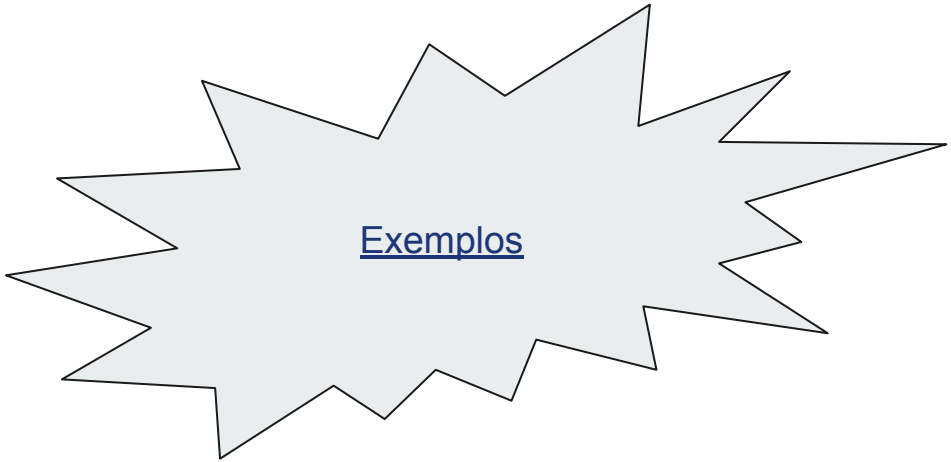
- **Invariante de Representação:** condição lógica que sempre deve valer para o objeto válido
- __init__: deve estabelecer o invariante
- Demais métodos: devem preservar o invariante após cada operação

Classes, Objetos e Atributos

- Invariante de Representação

```
1   class Conjunto:
2       def __init__(self, elementos=None):
3           self._vals = []
4           if elementos:
5               for e in elementos:
6                   if e not in self._vals:
7                       self._vals.append(e)  # invariante estabelecido
8       def insert(self, e):
9           if e not in self._vals:
10              self._vals.append(e)        # invariante preservado
11      def remove(self, e):
12          self._vals.remove(e)            # ValueError se não houver
```

Classes, Objetos e Atributos





Sobrescrevendo Métodos Reservados

Sobrescrevendo Métodos Reservados

- **_init_()** é um método reservado de Python.
- Um método reservado é disponibilizado pela linguagem e tem uma finalidade específica.
- Há outros métodos reservados:

__str__()
__add__() +
__sub__() -
__eq__() ==
__lt__() <

Sobrescrevendo Métodos Reservados

```
class Ponto(object):
    def _init_(self, x,y):
            self.x=x
            self.y=y

    def distancia(self,outro):
            deltaX=(self.x-outro.x)
            deltaY=(self.y-outro.y)
            return (deltaX**2+deltaY**2)**0.5
```

```
def _str_(self):
    return '('+str(self.x)+','+str(self.y)+')'
```



```
p=Ponto(3,4)

print(p) ⇒ (3,4)
```

Sobrescrevendo Métodos Reservados

```python
class Conjunto:
    def __init__(self, elementos=None):
        self._vals = []
        if elementos:
            for e in elementos:
                if e not in self._vals:
                    self._vals.append(e)  # invariante estabe
    def insert(self, e):
        if e not in self._vals:
            self._vals.append(e)        # invariante preserva
    def remove(self, e):
        self._vals.remove(e)            # ValueError se não
    def __contains__(self, e): return e in self._vals
    def __len__(self): return len(self._vals)
    def __repr__(self): return f'{str(self._vals)}'
    def __str__(self): return str(self._vals)
```

```python
c1 = Conjunto([1,2,3])
print(f'{c1=}')
c2 = Conjunto([1,2,3,4,5])
print(f'{c2=}')
c2.insert(c1)
print(c2)
print(f'{c2.__str__()=}')
print(f'{c2.__repr__()}')
```

```
c1=[1, 2, 3]
c2=[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, [1, 2, 3]]
c2.__str__()='[1, 2, 3, 4, 5, [1, 2, 3]]'
[1, 2, 3, 4, 5, [1, 2, 3]]
```

Sobrescrevendo Métodos Reservados

- __repr__ é o “retrato oficial” de um objeto em Python, voltado para desenvolvedores.
- A ideia é fornecer uma representação não ambígua e, quando possível, reconstruir o objeto.

Sobrescrevendo Métodos Reservados

```
1  c1 = Conjunto([1,2,3])
2  print(f'{c1=}')
3  c2 = Conjunto([1,2,3,4,5])
4  print(f'{c2=}')
5  c2.insert(c1)
6  print(c2)
7  print(f'{c2.__str__()=}')
8  print(f'{c2.__repr__()}')
```

```
class Conjunto:
    def __init__(self, elementos=None):
        self._vals = []
        if elementos:
            for e in elementos:
                if e not in self._vals:
                    self._vals.append(e)  # invariante estabe
    def insert(self, e):
        if e not in self._vals:
            self._vals.append(e)        # invariante preserva
    def remove(self, e):
        self._vals.remove(e)            # ValueError se não h
    def __contains__(self, e): return e in self._vals
    def __len__(self): return len(self._vals)
    def __repr__(self): return f'{str(self._vals)}'
    def __str__(self): return str(self._vals)
```

```
print(c1 in c2) #uso de __contains__
print([1,2,3] in c2) #uso de __contains__
print(len(c1)) #uso de __len__
print(len(c2)) #uso de __len__
```

```
True
False
3
6
```

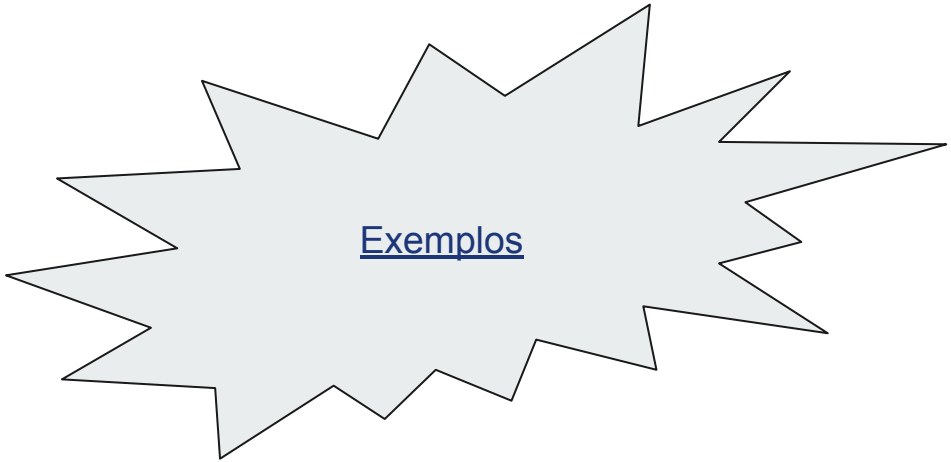
Sobrescrevendo Métodos Reservados

```
2  class Fracao(object):
3
4      def __init__(self, num,den):
5          assert type(num) == int and type(den) == int,'Erro: não é inteiro'
6          self.num=num
7          self.den=den
8
9      def __add__(self,outro):
10         somaDen=self.den*outro.den
11         somaNum=outro.den*self.num + self.den*outro.num
12         if(somaNum%somaDen==0):
13             somaNum=somaNum/somaDen
14             somaDen=1
15         return Fracao(somaNum,somaDen)
16
17     def __str__(self):
18         return str(self.num)+'/'+str(self.den)
19
```

```
f1=Fracao(3,4)
f2=Fracao(4,8)
print(f1,f2)
f3=f1+f2
print(f3)
```

```
3/4 4/8
40/32
```

Sobrescrevendo Métodos Reservados





Assertion & Exceptions

Assertion

- Utilizado para avaliar se determinada condição é necessária para a execução do código.
- Permite testar se tal condição é verdadeira. Se for falsa, dispara **AssertionError**
- No caso da condição ser falsa, uma mensagem de erro pode ser retornada.
- **assert** <condição> , <mensagem>

Assertion

```python
def ordenado(xs):
    assert all(xs[i] <= xs[i+1] for i in range(len(xs)-1)), "resultado não está ordenado"
    return xs
```

```
/tmp/ipython-input-366057078.py in ordenado(xs)
      1 def ordenado(xs):
----> 2     assert all(xs[i] <= xs[i+1] for i in
      3     return xs
      4 #ordenado([1,2,3])
      5 ordenado([4,3,2,1])

AssertionError: resultado não está ordenado
```

Assertion

```python
class Retangulo:
    def __init__(self, w, h):
        assert w > 0 and h > 0, f"medidas devem ser > 0; w={w}, h={h}"
        self.w, self.h = w, h

    def area(self):
        return self.w * self.h

r = Retangulo(10, 20)
print(f'{r.area()=}')
r = Retangulo(-10, 20)
print(f'{r.area()=}')
```

```
r.area()=200
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
/tmp/ipython-input-49977672.py in <cell line: 0>()
      9 r = Retangulo(10, 20)
     10 print(f'{r.area()=}')
---> 11 r = Retangulo(-10, 20)
     12 print(f'{r.area()=}')

/tmp/ipython-input-49977672.py in __init__(self, w, h)
      1 class Retangulo:
      2     def __init__(self, w, h):
----> 3         assert w > 0 and h > 0, f"medidas devem ser > 0; w={w}, h={h}"
      4         self.w, self.h = w, h
      5

AssertionError: medidas devem ser > 0; w=-10, h=20
```

Exception

- Indicam a ocorrência de um evento inesperado durante a execução do código.
- Tratam eventos cuja ocorrência não seja frequente mas que comprometem a execução do código.
- Assim, permitem aos programadores tratarem tais eventos de forma adequada.
- Os eventos podem estar relacionados a erros ou situações inesperadas.

Exception

```
1  L=[1,2,3]
2  print(f'{L[3]=}')
3
```

```
---------------------------------------------------------
IndexError                                Traceback
<ipython-input-1-00e491ad74c6> in <cell line: 2>()
      1 L=[1,2,3]
----> 2 print(f'{L[3]=}')

IndexError: list index out of range
```

```
1  L=[]
2  print(f'{10/len(L)}')
3
```

```
---------------------------------------------------------
ZeroDivisionError                         Traceback
<ipython-input-3-c8509ceec060> in <cell line: 2>()
      1 L=[]
----> 2 print(f'{10/len(L)}')

ZeroDivisionError: division by zero
```

```
1  print('x'/10)
2
```

```
---------------------------------------------------------------
TypeError                                 Traceback (most rece
<ipython-input-4-da56615c7243> in <cell line: 1>()
----> 1 print('x'/10)

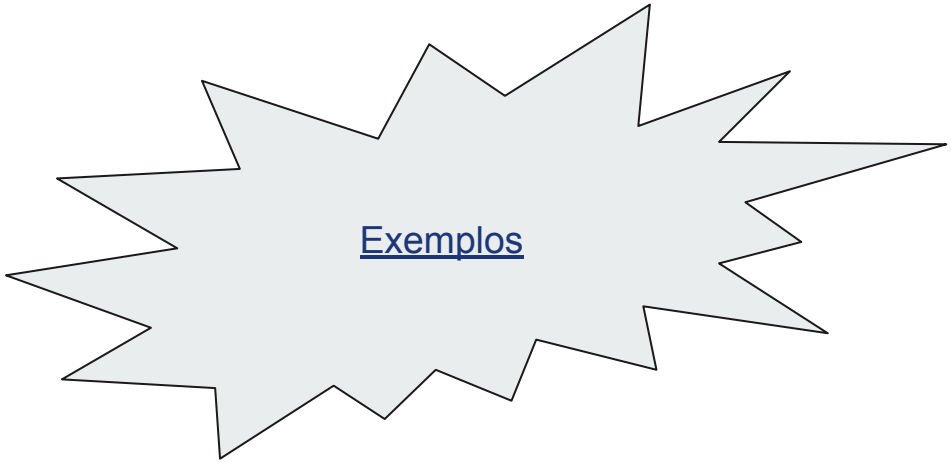
TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

Exception

- **Formas de lidar com eventos inesperados:**
  - try/exception
  - try/exception/else
  - try/finally

Exception

- **try** : habilita o tratamento da exceção ao conter o bloco de instruções que podem causar exceções.
- **exception** : determina possíveis causas para as exceções, podendo especificar um identificador para aquele tipo de erro.
- **else** : contém instruções a serem executadas, caso nenhuma exceção seja detectada no bloco try.
- **finally** : Um bloco finally define instruções a serem executadas independente da ocorrência ou não de exceções.

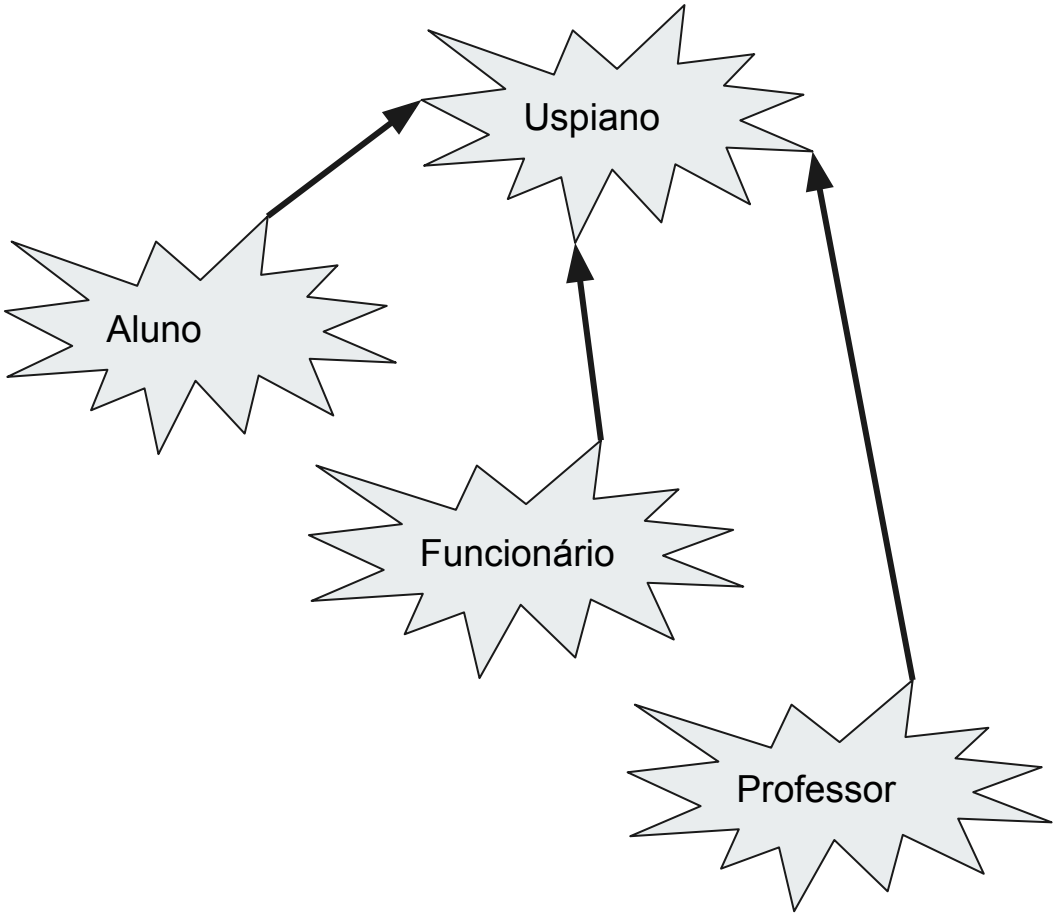

Exemplos



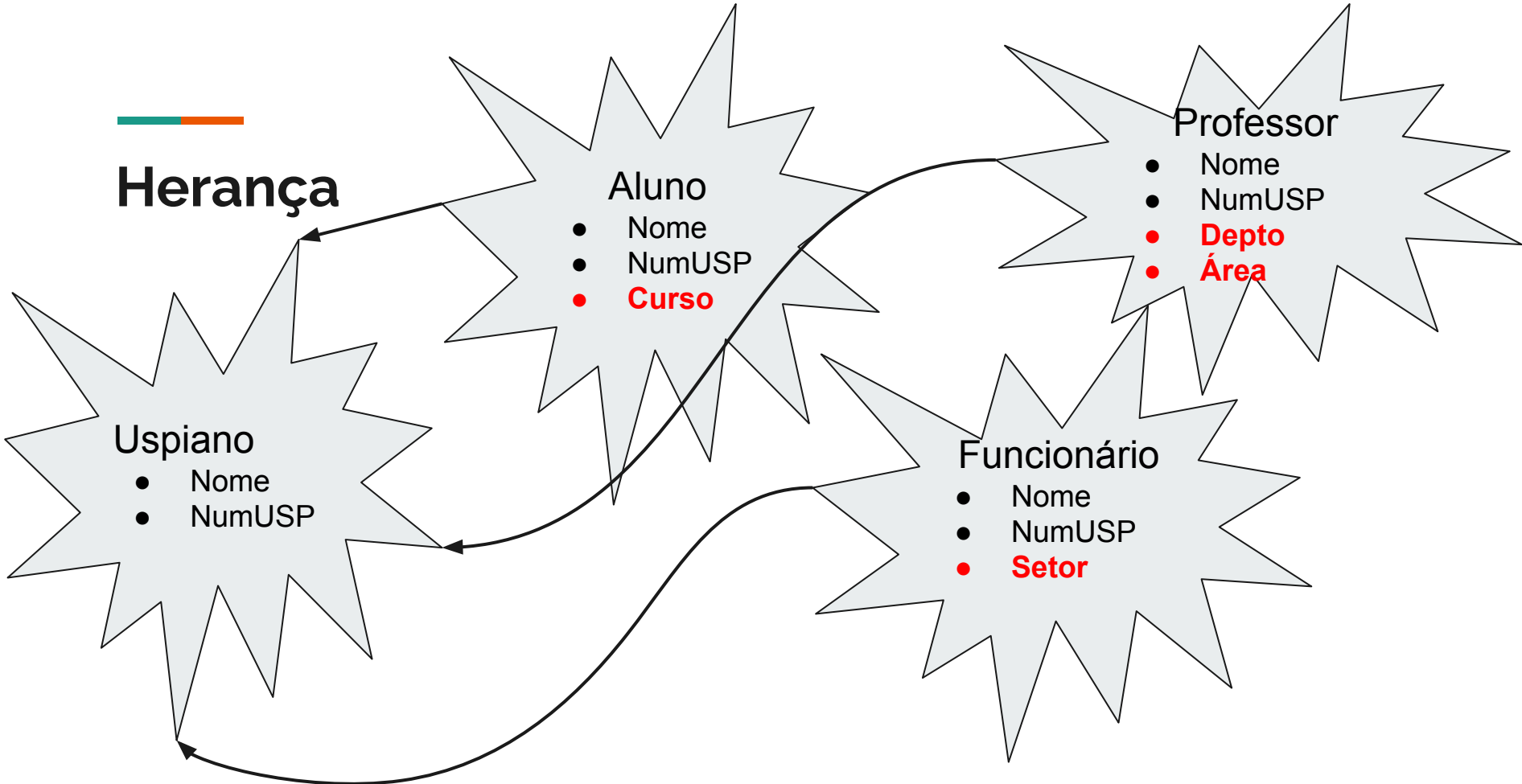
Herança

Herança





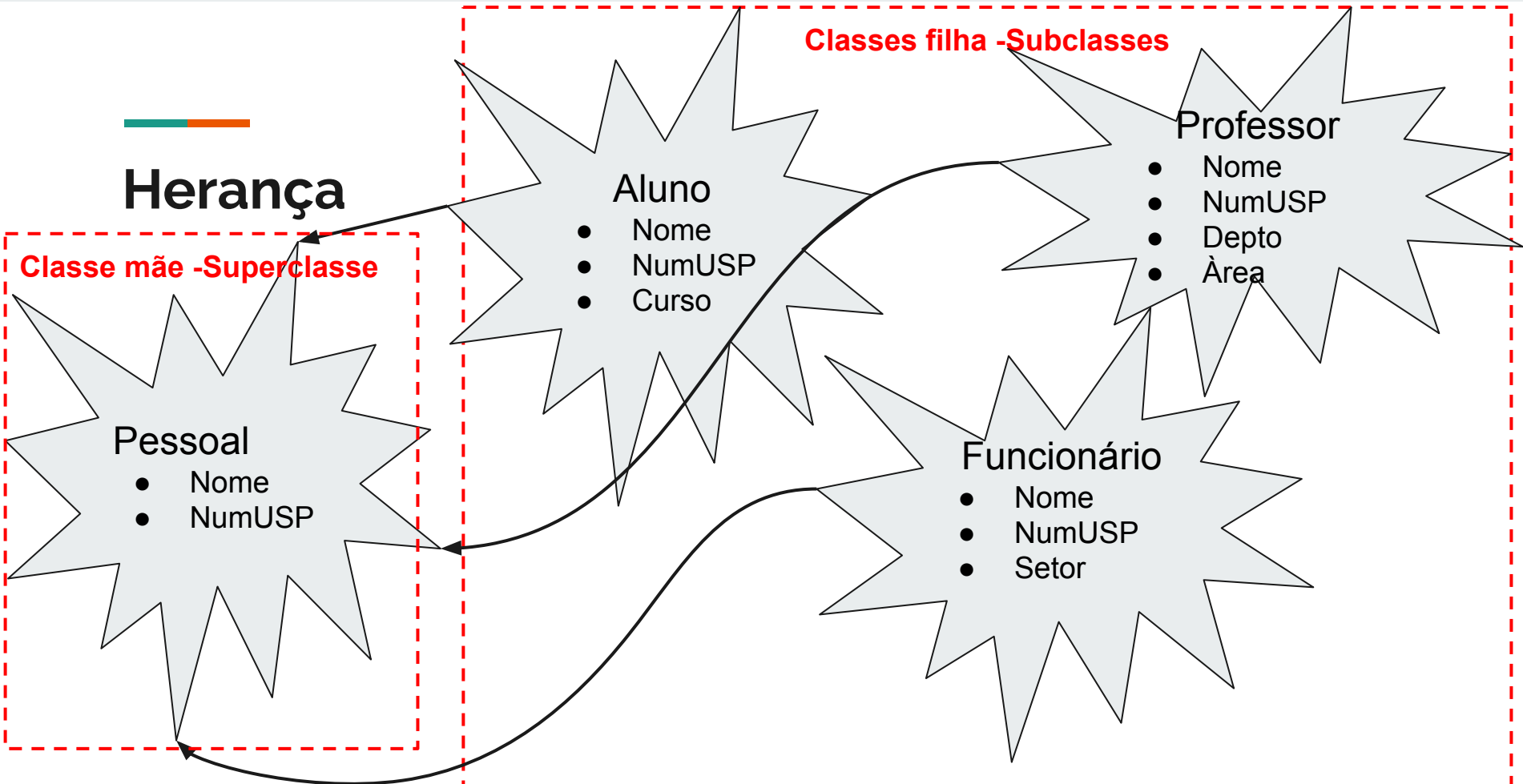
Herança


Aluno
Nome
NumUSP
Curso
Professor
Nome
NumUSP
Depto
Área
Uspiano
Nome
NumUSP
Funcionário
Nome
NumUSP
Setor

Herança

- Trata-se de uma forma bastante conveniente de relacionar grupos de abstrações.
- Torna-se possível estabelecer uma hierarquia onde classes filha herdam atributos das classes mãe.

Herança


Classes filha -Subclasses
Classe mãe -Superclasse
Aluno
Nome
NumUSP
Curso
Professor
Nome
NumUSP
Depto
Àrea
Pessoal
Nome
NumUSP
Funcionário
Nome
NumUSP
Setor

Herança

- Subclasse herda os atributos da superclasse: dados e métodos.
- Na subclasse é possível:
  - Adicionar novos dados e métodos
  - Sobrescrever métodos da superclasse.

Herança

```python
class Uspiano(object):

    def __init__(self,nome,numUSP):
        self.nome=nome
        self.numUSP= numUSP

    def set_nome(self,nome):
        self.nome=nome

    def get_nome(self):
        return self.nome

    def get_numUSP(self):
        return self.numUSP

    def __str__(self):
        return f'{self.nome},{self.numUSP}'
```

```python
class Aluno(Uspiano):

    def __init__(self,nome,numUSP,curso):
        Uspiano.__init__(self,nome,numUSP)
        self.curso=curso

    def set_curso(self,curso):
        self.curso = curso

    def set_nota(self,index,nota):
        self.provas[index]=nota


    def get_curso(self):
        return self.curso

    def __str__(self):
        return '\nAluno:'+Uspiano.__str__(self)+','\
    +self.curso
```

Herança

```
2   class Uspiano(object):
3
4       def __init__(self,nome,numUSP):
5           self.nome=nome
6           self.numUSP= numUSP
7
8       def set_nome(self,nome):
9           self.nome=nome
10
11      def get_nome(self):
12          return self.nome
13
14      def get_numUSP(self):
15          return self.numUSP
16
17      def __str__(self):
18          return f'{self.nome},{self.numUSP}'
```

```
46  class Funcionario(Uspiano):
47
48      def __init__(self,nome,numUSP,setor):
49          Uspiano.__init__(self,nome,numUSP)
50          self.setor=setor
51
52      def set_setor(self,setor):
53          self.setor = setor
54
55      def get_setor(self):
56          return self.setor
57
58      def __str__(self):
59          return '\nFuncionário:'+Uspiano.__str__(self)+\
60      ','+self.setor
```

Herança

```
2   class Uspiano(object):
3
4       def __init__(self,nome,numUSP):
5           self.nome=nome
6           self.numUSP= numUSP
7
8       def set_nome(self,nome):
9           self.nome=nome
10
11      def get_nome(self):
12          return self.nome
13
14      def get_numUSP(self):
15          return self.numUSP
16
17      def __str__(self):
18          return f'{self.nome},{self.numUSP}'
```

```
62  class Professor(Uspiano):
63
64      def __init__(self,nome,numUSP,dept,area):
65          Uspiano.__init__(self,nome,numUSP)
66          self.dept = dept
67          self.area = area
68
69      def set_dept(self,dept):
70          self.dept = dept
71
72      def get_dept(self):
73          return self.dept
74
75      def set_area(self,area):
76          self.area = area
77
78      def get_area(self):
79          return self.area
80
81      def __str__(self):
82          return '\nProfessor:'+Uspiano.__str__(self)+','\
83      +self.dept+','+self.area
```

Herança

```
a=Aluno('Joao',1,'BCC')
print(a)
a.set_curso('Estatística')
print(a)
f=Funcionario('Pedro',2,'STI')
p=Professor('Claudio',3,'SSC','Otimização')
print(a,f,p)
```

```
Aluno:Joao,1,BCC

Aluno:Joao,1,Estatística

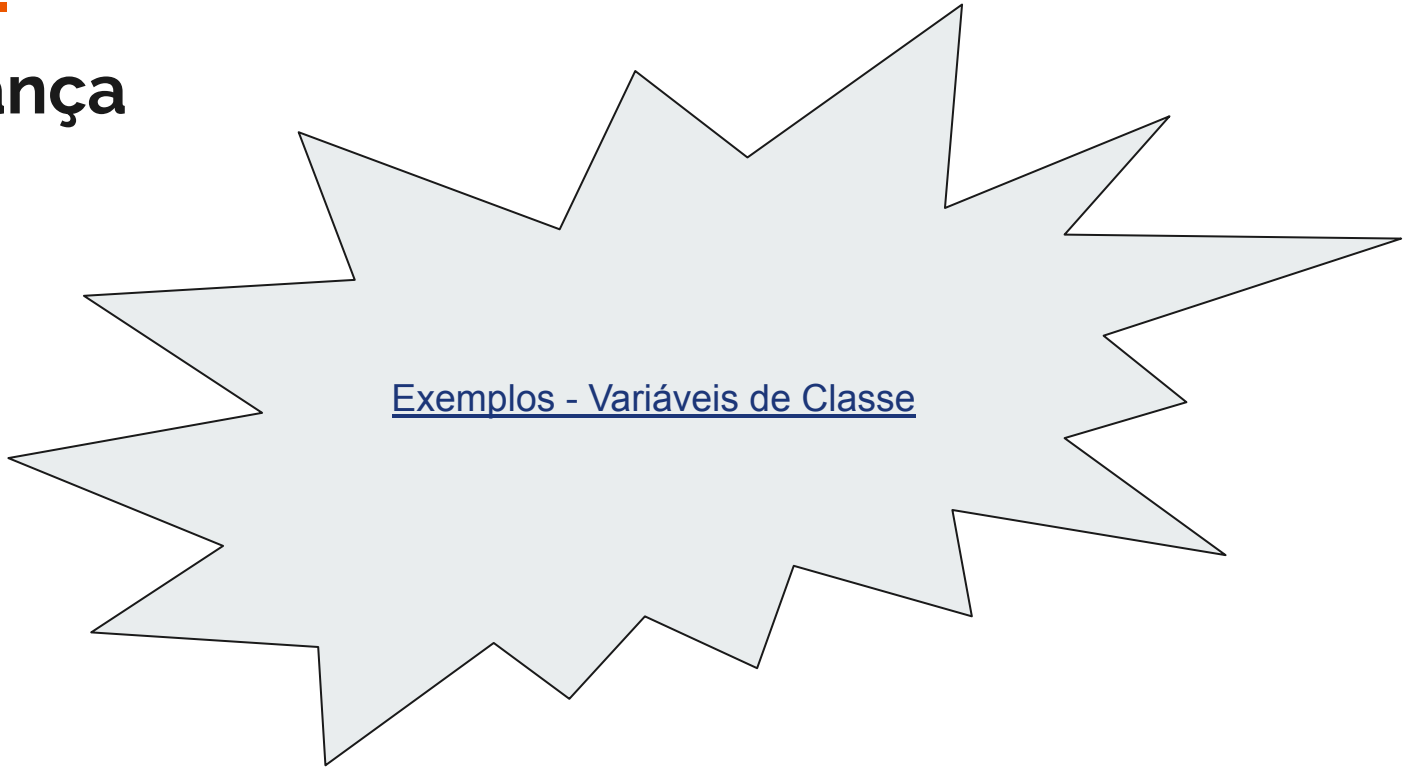
Aluno:Joao,1,Estatística
Funcionário:Pedro,2,STI
Professor:Claudio,3,SSC,Otimização
```

```
21  class Aluno(Uspiano):
22
23      def __init__(self,nome,numUSP,curso):
24          Uspiano.__init__(self,nome,numUSP)
25          self.curso=curso
26
27      def set_curso(self,curso):
28          self.curso = curso
29
30      def set_nota(self,index,nota):
31          self.provas[index]=nota
32
33
34      def get_curso(self):
35          return self.curso
36
37      def __str__(self):
38          return '\nAluno:'+Uspiano.__str__(self)+','\
39      +self.curso
```

```
46  class Funcionario(Uspiano):
47
48      def __init__(self,nome,numUSP,setor):
49          Uspiano.__init__(self,nome,numUSP)
50          self.setor=setor
51
52      def set_setor(self,setor):
53          self.setor = setor
54
55      def get_setor(self):
56          return self.setor
57
58      def __str__(self):
59          return '\nFuncionário:'+Uspiano.__str__(self)+\
60      ','+self.setor
```

```
62  class Professor(Uspiano):
63
64      def __init__(self,nome,numUSP,dept,area):
65          Uspiano.__init__(self,nome,numUSP)
66          self.dept = dept
67          self.area = area
68
69      def set_dept(self,dept):
70          self.dept = dept
71
72      def get_dept(self):
73          return self.dept
74
75      def set_area(self,area):
76          self.area = area
77
78      def get_area(self):
79          return self.area
80
81      def __str__(self):
82          return '\nProfessor:'+Uspiano.__str__(self)+','\
83      +self.dept+','+self.area
```

Herança





Encapsulamento em Python

Encapsulamento em Python

- Encapsulamento: Prática de esconder detalhes de implementação e expôr uma interface estável.
- Linguagens orientadas a objetos (Java, C++, etc.) controlam ou restringem o acesso a atributos das classes.
- Nesse caso, os atributos da classe passam a ser classificados como público (*public*), privado (*private*) ou protegido (*protected*).
- Não há mecanismos em Python que impeçam de fato o acesso a qualquer variável ou método de uma classe.

Encapsulamento em Python

- Objetivo: separar o que um objeto faz (sua API pública) de como ele faz (detalhes internos).
- Benefícios: legibilidade, segurança contra uso incorreto, facilidade de manutenção e evolução sem quebrar usuários da classe.
- Em Python, não há “private/protected” rígido como em Java/C++; ao invés disso, usamos convenções e alguns mecanismos que desencorajam o acesso direto.

Encapsulamento em Python

- Público: <>.nome
  - Pode ser usado por código externo sem ressalvas.
- “Protegido” (convenção) :< >._nome
  - “Uso interno / para subclasses”. O interpretador não bloqueia, mas sinaliza: **não dependa disso.**
- “Privado” com name mangling: <>.__nome
  - Python renomeia internamente para _Classe__nome, reduzindo colisões com subclasses e desencorajando acesso acidental.

Encapsulamento em Python

- Todos os atributos de uma classe em Python são públicos por definição.
- Atributos públicos (public) podem ser acessados fora da classe.

```
2  class Ponto(object):
3
4      def __init__(self,x,y):
5          self.x = x
6          self.y = y
7
8      def distancia(self,outro):
9          deltaX=(self.x-outro.x)
10         deltaY=(self.y-outro.y)
11         return (deltaX**2+deltaY**2)**0.5
12     def __str__(self):
13         return '('+str(self.x)+','+str(self.y)+')
14
15 p=Ponto(5,4)
16 print(f'{p!s}')
17 p.x=3
18 p.y=2
19 print(f'{p!s}')
20 saida=p.__str__()
21 print(saida)
```

```
(5,4)
(3,2)
(3,2)
```

Encapsulamento em Python

- Python permite simular acessos do tipo protegido ou privado através da adição de prefixos aos nomes de variáveis e métodos de uma classe.
- Atributos **protegidos (protected)** de uma classe podem ser acessados **dentro da própria classe e pelas classes filhas.**
- Atributos protegidos não podem ser acessados a partir de outros ambientes.
- Utiliza-se 1 underscore: _<nomeAtributo>

```python
class Ponto(object):

    def __init__(self,x,y):
        self._x = x
        self._y = y

    def distancia(self,outro):
        deltaX=(self._x-outro.x)
        deltaY=(self._y-outro.y)
        return (deltaX**2+deltaY**2)**0.5
    def __str__(self):
        return '('+str(self._x)+','+str(self._y)+')'

p=Ponto(5,4)
print(f'{p!s}')
p.x=3
p.y=2
print(f'{p!s}')
saida=p.__str__()
print(saida)
```

```
(5,4)
(5,4)
(5,4)
```

```python
class PontoColorido(Ponto):
    def __init__(self, x, y, cor):
        super().__init__(x, y)
        self.cor = cor

    def deslocar(self, dx, dy):
        # Subclasse acessa atributos "protegidos" por convenção
        self._x += dx
        self._y += dy


p = Ponto(5, 4)
print(f'Antes-{p!s}')   # -> (5,4)   (usa __str__)
p.x = 3
p.y = 2
print(f'Depois-{p!s}')   # -> (3,2)

pc = PontoColorido(1, 1, "azul")
pc.deslocar(2, -2)
print(f'{pc!s} {pc.cor}')  # -> (3,0) azul
```

```
Antes-(5,4)
Depois-(5,4)
(3,-1) azul
3.0 -1.0
```

```python
print("TUDO É UMA ILUSÃO")
print(f'Antes: {p._x}, {p._y}')  # evitar fora da classe/subclasses
p._x = 10
p._y = 20
print(f'Depois: {p._x}, {p._y}')  # evitar fora da classe/subclasses
print(f'Antes: {pc._x}, {pc._y}')  # evitar fora da classe/subclasses
pc._x = 3
pc._y = 2
print(f'Depois: {pc._x}, {pc._y}')  # evitar fora da classe/subclasses
```

```
TUDO É UMA ILUSÃO
Antes: 5.0, 4.0
Depois: 10, 20
Antes: 3.0, -1.0
Depois: 3, 2
```

Atributos Privados

- Atributos privados têm seu acesso efetivamente impedido a partir de outros ambientes.
- Tentativas de acesso externo resultam em erro.
- Utiliza-se dois underscore: _ _<nomeAtributo>

Atributos Privados

```python
class Cofre:
    def __init__(self, codigo):
        self.__codigo = str(codigo)     # vira

    def abrir(self, tentativa):
        return str(tentativa) == self.__codigo

cofre = Cofre(1234)
print(cofre.__codigo)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most rece
/tmp/ipython-input-600515828.py in <cell line: 0>()
      7
      8 cofre = Cofre(1234)
----> 9 print(cofre.__codigo)                # AttributeError

AttributeError: 'Cofre' object has no attribute '__codigo'
```

- “Privado” com name mangling: <>.__nome
  - **Python renomeia internamente para _Classe__nome**, reduzindo colisões com subclasses e desencorajando acesso acidental.

```python
print(cofre._Cofre__codigo)
print(cofre.abrir(1234))
```

```
1234
True
```

@property

- Permite expor um atributo como se fosse público, mas com lógica de acesso por trás (validação, cálculo sob demanda, conversões, cache, etc.).
- Assim você mantém a API simples (obj.preco) sem abrir mão do encapsulamento
- O encapsulamento é definido via getter/setter

@property

- Permite expor um atributo como se fosse público, mas com lógica de acesso por trás (validação, cálculo sob demanda, conversões, cache, etc.).
- Assim você mantém a API simples (obj.preco) sem abrir mão do encapsulamento
- O encapsulamento é definido via getter/setter

@property

- -> float é uma anotação de tipo (type hint) que indica o tipo de retorno esperado da função.
- No código def preco(self) -> float:, significa: “esta função deve retornar um float”

```python
class Produto:
    def __init__(self, quantidade, preco: float):
        self._preco = 0.0       # atributo de apoio (convencionalmente com _)
        self.preco = preco      # passa pelo setter
        self._quantidade=0.0
        self.quantidade = quantidade

    @property
    def preco(self) -> float:  # getter
        """Preço atual do produto (não negativo)."""
        return self._preco

    @property
    def quantidade(self) -> float:  # getter
        """Quantidade atual do produto (não negativo)."""
        return self._quantidade

    @preco.setter
    def preco(self, valor: float):  # setter
        if valor < 0:
            raise ValueError("Preço não pode ser negativo.")
        self._preco = float(valor)

    @quantidade.setter
    def quantidade(self, valor: float):  # setter
        if valor < 0:
            raise ValueError("Quantidade não pode ser negativo.")
        self._quantidade = float(valor)


    @preco.deleter
    def preco(self):  # deleter (raro, mas existe)
        del self._preco

```

```python
class Produto:
    def __init__(self, quantidade, preco: float):
        self._preco = 0.0      # atributo de apoio (convencionalmente com _)
        self.preco = preco     # passa pelo setter
        self._quantidade=0.0
        self.quantidade = quantidade

    @property
    def preco(self) -> float:  # getter
        """Preço atual do produto (não negativo)."""
        return self._preco

    @property
    def quantidade(self) -> float:  # getter
        """Quantidade atual do produto (não negativo)."""
        return self._quantidade

    @preco.setter
    def preco(self, valor: float):  # setter
        if valor < 0:
            raise ValueError("Preço não pode ser negativo.")
        self._preco = float(valor)

    @quantidade.setter
    def quantidade(self, valor: float):  # setter
        if valor < 0:
            raise ValueError("Quantidade não pode ser negativo.")
        self._quantidade = float(valor)


    @preco.deleter
    def preco(self):  # deleter (raro, mas existe)
        del self._preco

```

```python
p = Produto(10,10)
print(p.preco,p.quantidade)   # 10.0  (chama o getter)
p.preco = 12.5   #       (chama o setter)
print(p.preco,p.quantidade)   # 12.5
del p.preco    #       (chama o deleter)
print(p.quantidade)
print(p.preco)
```

```
10.0 10.0
12.5 10.0
10.0
---------------------------------------------------------------------------
AttributeError                            Traceback (most rece
/tmp/ipython-input-38240599.py in <cell line: 0>()
     39 del p.preco    #       (chama o deleter)
     40 print(p.quantidade)
---> 41 print(p.preco)

/tmp/ipython-input-38240599.py in preco(self)
      9     def preco(self) -> float:  # getter
     10         """Preço atual do produto (não negativo)."""
---> 11         return self._preco
     12
     13     @property

AttributeError: 'Produto' object has no attribute '_preco'
```

@property

- Quando você usa @property sobre **self.preco** com **self.preco=preco**, faz uma chamada ao setter.
- Isso faz o Python chamar o mesmo setter de novo, que chama de novo, e assim por diante — até estourar um RecursionError

```python
class Produto:
    def __init__(self, preco):
        self.preco = preco  # chama o setter

    @property
    def preco(self):
        return self.preco   # ❌ chama o getter de novo (recursão no getter)

    @preco.setter
    def preco(self, valor):
        if valor < 0:
            raise ValueError("Preço não pode ser negativo.")
        self.preco = float(valor)  # ❌ chama o setter de novo (recursão no setter)
p = Produto(10)
print(p.preco)
```

```
---------------------------------------------------------------------------
RecursionError                            Traceback (most recent
/tmp/ipython-input-1843416838.py in <cell line: 0>()
     12             raise ValueError("Preço não pode ser negativ
     13         self.preco = float(valor)  # ❌ chama o setter d
---> 14 p = Produto(10)
     15 print(p.preco)

                              2 frames
... last 1 frames repeated, from the frame below ...

/tmp/ipython-input-1843416838.py in preco(self, valor)
     11         if valor < 0:
     12             raise ValueError("Preço não pode ser negativ
---> 13         self.preco = float(valor)  # ❌ chama o setter d
     14 p = Produto(10)
     15 print(p.preco)

RecursionError: maximum recursion depth exceeded
```

@property

- Temos dois erros:
  - **getter** retorna self.preco (chama o getter outra vez)
  - **setter** atribui self.preco (chama o setter de novo).
- Em ambos os casos, é recursão infinita.

```python
class Produto:
    def __init__(self, preco):
        self.preco = preco  # chama o setter

    @property
    def preco(self):
        return self.preco   # ❌ chama o getter de novo (recursão no getter)

    @preco.setter
    def preco(self, valor):
        if valor < 0:
            raise ValueError("Preço não pode ser negativo.")
        self.preco = float(valor)  # ❌ chama o setter de novo (recursão no setter)
p = Produto(10)
print(p.preco)
```

```
---------------------------------------------------------------------------
RecursionError                            Traceback (most recent
/tmp/ipython-input-1843416838.py in <cell line: 0>()
     12             raise ValueError("Preço não pode ser negativ
     13         self.preco = float(valor)  # ❌ chama o setter d
---> 14 p = Produto(10)
     15 print(p.preco)

                                 2 frames
... last 1 frames repeated, from the frame below ...

/tmp/ipython-input-1843416838.py in preco(self, valor)
     11         if valor < 0:
     12             raise ValueError("Preço não pode ser negativ
---> 13         self.preco = float(valor)  # ❌ chama o setter d
     14 p = Produto(10)
     15 print(p.preco)

RecursionError: maximum recursion depth exceeded
```

@property

- Solução: atributo de apoio (backing field)
  - Crie um atributo “de verdade” com outro nome (geralmente com _), e faça o getter/setter operarem sobre ele, não sobre a property.

```python
class Produto:
    def __init__(self, preco):
        self._preco = 0.0           # atributo de apoio
        self.preco = preco          # usa o setter, com validação

    @property
    def preco(self):
        return self._preco          # lê o atributo de apoio

    @preco.setter
    def preco(self, valor):
        if valor < 0:
            raise ValueError("Preço não pode ser negativo.")
        self._preco = float(valor)  # grava no atributo de apoio

p = Produto(10)
print(p.preco)
```

```
10.0
```

@property

- Herança e @property
  - Você pode sobrescrever um getter/setter em subclasses para refinar regras..

```python
class Conta:
    def __init__(self, saldo=0):
        self._saldo = 0.0
        self.saldo = saldo

    @property
    def saldo(self):
        return self._saldo

    @saldo.setter
    def saldo(self, v):
        if v < 0:
            raise ValueError("Saldo não pode ser negativo.")
        self._saldo = float(v)

class ContaComTarifa(Conta):
    @Conta.saldo.setter
    def saldo(self, v):             # refina a validação

        super(ContaComTarifa, type.(self))saldo.__set__(self, v)  # chama setter base
        #Conta.saldo.__set__(self,v)

        # aplica uma taxa fixa de abertura, por exemplo
        if not hasattr(self, "_taxa_aplicada"):
            self._saldo -= 5.0
            self._taxa_aplicada = True
```

```python
c1 =Conta(10000)
print(c1.saldo)
c2=ContaComTarifa(10000)
print(c2.saldo)
```

```
10000.0
9995.0
```

__slots__

- Lista/tupla de nomes de atributos permitidos na instância.
- Quando você define __slots__, o Python não cria o dicionário de instância (__dict__) por padrão.
- Sem __slots__, toda instância tem um __dict__ onde qualquer atributo novo pode ser criado a qualquer momento.

```python
class Vetor2D:
    # somente os atributos (x,y) poderão existir
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = float(x)
        self.y = float(y)
```

```python
v = Vetor2D(1, 2)
print(v.x, v.y)
```

```
1.0 2.0
```

```python
v.z = 3
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipython-input-1316107358.py in <cell line: 0>()
----> 1 v.z = 3          # AttributeError: 'Vetor2D' object has no attribute 'z'

AttributeError: 'Vetor2D' object has no attribute 'z'
```

__slots__

- Logo, restringe a criação de atributos fora dos listados (protege contra “vazamentos”);
- Economiza memória (sem __dict__ por instância);
- Pode acelerar acesso a atributos.

```python
class Vetor2D:
    # somente os atributos (x,y) poderão existir
    __slots__ = ("x", "y")

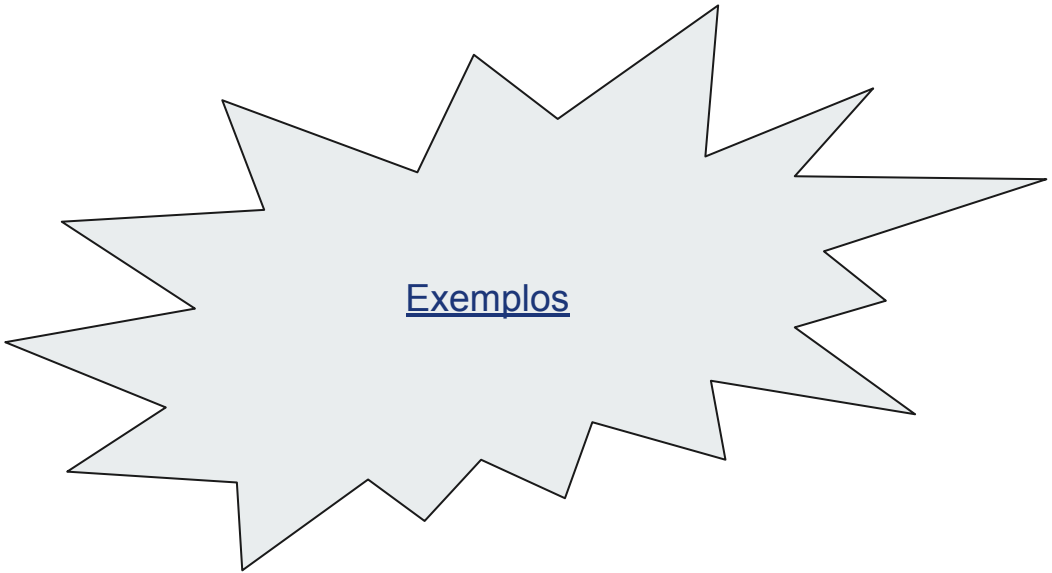
    def __init__(self, x, y,z):
        self.x = float(x)
        self.y = float(y)
        self.z = float(z)

v = Vetor2D(1, 2, 3)
print(v.x, v.y,v.z)   # 1.0 2.0
```

```
---------------------------------------------------------------
AttributeError                            Traceback (most rec
/tmp/ipython-input-3711374479.py in <cell line: 0>()
      8         self.z = float(z)
      9
---> 10 v = Vetor2D(1, 2, 3)
     11 print(v.x, v.y,v.z)   # 1.0 2.0
     12

/tmp/ipython-input-3711374479.py in __init__(self, x, y, z)
      6         self.x = float(x)
      7         self.y = float(y)
----> 8         self.z = float(z)
      9
     10 v = Vetor2D(1, 2, 3)

AttributeError: 'Vetor2D' object has no attribute 'z'
```


Exemplos



Polimorfismo & Classes Abstratas

Polimorfismo

- Polimorfismo = “muitas formas”.
- Uma mesma interface (mesmo nome de método/função) com múltiplas implementações.
- Vantagem: você programa contra a interface, não contra implementações concretas → código mais flexível, testável e extensível.

Polimorfismo

```python
def processar_pagamento(meio, valor):
    # Não importa a classe; basta ter .pagar(valor)
    meio.pagar(valor)

class CartaoCredito:
    def pagar(self, valor): print(f"[Cartão] pago R${valor:.2f}")

class Pix:
    def pagar(self, valor): print(f"[Pix] pago R${valor:.2f}")

class Boleto:
    def pagar(self, valor): print(f"[Boleto] pago R${valor:.2f}")

for m in (CartaoCredito(), Pix(), Boleto()):
    processar_pagamento(m, 99.9)
```

```
[Cartão] pago R$99.90
[Pix] pago R$99.90
[Boleto] pago R$99.90
```

Polimorfismo

- Protocol é um recurso de tipagem estática do Python que permite definir interfaces estruturais que utilizam contratos baseados no que um objeto sabe fazer, não no que ele herda.
- Protocol permite descrever a forma que um objeto deve ter (métodos/atributos esperados) para ser aceito por uma função, habilitando checagem **sem exigir herança..**

```python
from typing import Protocol

class Pagavel(Protocol):
    def pagar(self, valor: float) -> None: ...

def processar(p: Pagavel, valor: float) -> None:
    p.pagar(valor)

class Cartao:
    def pagar(self, valor: float) -> None:
        print(f"Cartão pagou R${valor:.2f}")

class Pix:
    def pagar(self, valor: float) -> None:
        print(f"Pix pagou R${valor:.2f}")

processar(Cartao(), 99.9)  # ok
processar(Pix(), 99.9)     # ok
c = Cartao()
p = Pix()
processar(c, 100.0)         # ok
processar(p, 100.0)         # ok
# Uma classe sem .pagar(...) acusará erro no checker estático.
```

```
Cartão pagou R$99.90
Pix pagou R$99.90
Cartão pagou R$100.00
Pix pagou R$100.00
```

Polimorfismo

- Protocol é um recurso de tipagem estática do Python que permite definir interfaces estruturais que utilizam contratos baseados no que um objeto sabe fazer, não no que ele herda.
- Protocol permite descrever a forma que um objeto deve ter (métodos/atributos esperados) para ser aceito por uma função, habilitando checagem **sem exigir herança..**

```python
from typing import Protocol

class TemArea(Protocol):
    def area(self) -> float: ...

class Triangulo:
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self) -> float: return (self.b * self.h) / 2

class Quadrilatero:
    def __init__(self, l1, l2): self.l1, self.l2 = l1, l2
    def area(self) -> float: return (self.l1 * self.l2)

def imprimir_area(forma: TemArea):
    print(f"Área = {forma.area():.2f}")
```

```
Área = 6.00
Área = 12.00
```

Polimorfismo

- Polimorfismo do Python “de fábrica”: utilizamos polimorfismo ao sobrecarregar operadores

```python
class Conjunto:
    def __init__(self, elementos=None):
        self._vals = []
        if elementos:
            for e in elementos:
                if e not in self._vals:
                    self._vals.append(e)  # invariante estabe
    def insert(self, e):
        if e not in self._vals:
            self._vals.append(e)        # invariante preserva
    def remove(self, e):
        self._vals.remove(e)            # ValueError se não h
    def __contains__(self, e): return e in self._vals
    def __len__(self): return len(self._vals)
    def __repr__(self): return f'{str(self._vals)}'
    def __str__(self): return str(self._vals)
```

```python
print(c1 in c2) #uso de __contains__
print([1,2,3] in c2) #uso de __contains__
print(len(c1)) #uso de __len__
print(len(c2)) #uso de __len__
```

```
True
False
3
6
```

Classes Abstratas

- Define um contrato (métodos/atributos que devem existir), mas não pode ser instanciada se ainda tiver membros abstratos pendentes.
- Classes abstratas devem ser usadas para padronizar interfaces (plugins/drivers/backends), reduzir acoplamento e capturar invariantes do design.
- Python por default não fornece classes abstratas, deve-se importar a Abstract Base Classes (ABCs)

```
from abc import ABC
```

Classes Abstratas

- As classes abstratas possuem um ou mais métodos do tipo abstrato.
- Um método abstrato é declarado, mas não é implementado.

```
@abstractmethod
def precisa_implementar(self):
    pass
```

- O decorador `@abstractmethod` torna o método abstrato.

```python
from abc import ABC, abstractmethod
from math import pi

class Forma(ABC):
    @abstractmethod
    def area(self) -> float:
        """Retorna a área da forma."""
        ...

    def descricao(self) -> str:
        # Métodos concretos podem depender de abstratos
        return f"Área = {self.area():.2f}"

class Retangulo(Forma):
    def __init__(self, w: float, h: float):
        self.w, self.h = w, h
    def area(self) -> float:
        return self.w * self.h

class Circulo(Forma):
    def __init__(self, r: float):
        self.r = r
    def area(self) -> float:
        return pi * (self.r ** 2)

print(Retangulo(2, 3).descricao())  # Área = 6.00
print(Circulo(1).descricao())       # Área = 3.14
f = Forma()             #  TypeError: não pode instanciar c
```

```
Área = 6.00
Área = 3.14
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-3696455780.py in <cell line: 0>()
     26 print(Retangulo(2, 3).descricao())  # Área = 6.00
     27 print(Circulo(1).descricao())       # Área = 3.14
---> 28 f = Forma()             #  TypeError: não pode instanciar classe abstrata

TypeError: Can't instantiate abstract class Forma without an implementation for abstract method 'area'
```

Classes Abstratas

